

TrainerBottomNav.js — Fundamentos

Explicación detallada de la barra de navegación del entrenador (GymBro App)

Es la barra de navegación inferior fija que ven los **entrenadores** en varias pantallas (buscar_usuario_trainer, editar_rutina_trainer, mi_cuenta_trainer, ver_datos_trainer). Conceptualmente es un “tab bar” hecho a mano, sin usar el Bottom Tabs Navigator de React Navigation.

1. Las props (línea 8)

export default function TrainerBottomNav({ activeScreen, isLandscape, navigation }) {
Recibe tres props, y cada pantalla que la usa se las pasa explícitamente (ejemplo en buscar_usuario_trainer.js:172-176):

```
<TrainerBottomNav  
  activeScreen="search"  
  isLandscape={isLandscape}  
  navigation={navigation}  
/>
```

- **activeScreen**: un string ("search" o "account") que cada pantalla define manualmente para decirle a la barra “yo soy esta pestaña, resáltame a mí”. No lo calcula el componente solo — cada screen se auto-identifica al usarlo.
- **isLandscape**: viene de useResponsiveLayout() en la pantalla padre y se reenvía acá para ajustar la altura de la barra en horizontal.
- **navigation**: el objeto de navegación de React Navigation, para poder llamar navigation.navigate(...) y cambiar de pantalla. La pantalla padre lo recibe como prop (porque ella está registrada en un Stack.Screen) y se lo **reenvía** a este componente hijo, que no está directamente registrado como ruta.

Punto importante: TrainerBottomNav no usa ningún hook (useAuth, useResponsiveLayout, etc.) — todo lo que necesita lo recibe por props. Esto lo hace **fácil de testear y predecible**: dado el mismo conjunto de props, siempre renderiza lo mismo.

2. Estructura: dos botones de navegación (líneas 16-60)

Hay dos TouchableOpacity, uno por cada sección a la que puede ir el entrenador:

```
<TouchableOpacity onPress={() => navigation.navigate('buscar_usuario_trainer')}>  
  <Text style={[styles.navIcon, activeScreen === 'search' && styles.active]}>■</Text>  
  <Text style={[styles.navText, activeScreen === 'search' && styles.active]}>Buscar Usuario</Text>  
</TouchableOpacity>  
  
<TouchableOpacity onPress={() => navigation.navigate('mi_cuenta_trainer')}>  
  <Text style={[styles.navIcon, activeScreen === 'account' && styles.active]}>■</Text>  
  <Text style={[styles.navText, activeScreen === 'account' && styles.active]}>Mi Cuenta</Text>  
</TouchableOpacity>
```

Cada botón tiene la misma estructura repetida (ícono + texto), y ambos siguen el mismo patrón:

- **Navegación explícita:** navigation.navigate('nombre_de_ruta'). Estas rutas (buscar_usuario_trainer, mi_cuenta_trainer) son las mismas que están registradas en el grupo “Entrenador” de app/index.tsx, así que solo existen (y solo se puede navegar a ellas) cuando user.isTrainer es true.

- **Resaltado condicional:** `activeScreen === 'search' && styles.active` — el mismo patrón de “estilo condicional con &&” que en `login.js/signup.js`. Si el string que le pasó la pantalla coincide con el que identifica a ese botón, se le agrega `styles.active` (color amarillo `#FFC107`) tanto al ícono como al texto. Si no coincide, quedan en gris (`#666666`, el color por defecto de `navIcon/navText`).
- Los “íconos” no son un ícono de una librería (a diferencia de `CustomInput` que usa `Ionicons`) — son **caracteres Unicode dentro de un Text**: `🔍` (lupa) y `♟` (peón de ajedrez, usado como ícono de persona/cuenta de forma estilizada). Es una solución liviana que evita depender de una librería de íconos para esto en particular.

3. Por qué `activeScreen` es un string manual y no se calcula solo

Fijate que el componente **no sabe en qué pantalla está montado** — no usa `useRoute()` de `React Navigation` ni nada equivalente para autodetectarlo. Cada pantalla se lo dice explícitamente vía prop. Esto es una decisión de diseño simple: en vez de que `TrainerBottomNav` tenga que comparar la ruta actual con las rutas registradas (lo cual acoplaría el componente a los nombres exactos de las screens), delega esa responsabilidad al padre. Es un poco más repetitivo (cada pantalla escribe `activeScreen="search"` o `activeScreen="account"`), pero mantiene el componente **desacoplado** de la lógica de rutas.

4. Los estilos (líneas 65-100)

```
bottomNav: {
  minHeight: '8%',
  backgroundColor: '#1D1D1D',
  borderTopWidth: 1,
  borderTopColor: '#292929',
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-around',
},
bottomNavLandscape: {
  minHeight: 56,
},
```

- `minHeight: '8%'` en modo portrait: la barra ocupa un porcentaje de la altura de pantalla, escalando con dispositivos de distinto tamaño.
- `bottomNavLandscape: { minHeight: 56 }`: en horizontal, un porcentaje sería demasiado alto (menos altura total disponible), así que se fija un valor absoluto en píxeles en su lugar — mismo patrón de “override con estilo condicional” que en `login.js` (`isLandscape && styles.cardLandscape`).
- `borderTopWidth/borderTopColor`: una línea sutil que separa visualmente la barra del contenido de arriba.
- `flexDirection: 'row' + justifyContent: 'space-around'`: los dos botones se reparten el ancho disponible de forma pareja.
- `navItem: { flex: 1, ... }`: cada botón ocupa el mismo espacio (`flex 1` cada uno = 50% del ancho, ya que son dos).
- `active: { color: '#FFC107' }`: un solo estilo compartido para “resaltado”, reutilizado tanto en `navIcon` como en `navText` mediante arrays de estilos (`[styles.navIcon, activeScreen === '...' && styles.active]`) — `React Native` permite combinar varios objetos de estilo en un array, y los combina en orden (el último gana si hay conflicto de propiedades).

5. Comparación con login.js/signup.js

Es la primera vez que aparece un componente que **recibe navigation como prop** en vez de recibirlo automáticamente de React Navigation. En login.js, navigation le llega solo porque la pantalla está registrada como Stack.Screen. Acá, como TrainerBottomNav es un componente hijo (no una ruta), no recibe navigation automáticamente — por eso cada pantalla debe **reenviárselo explícitamente** como prop.

Resumen para el oral

“TrainerBottomNav es una barra de navegación inferior hecha a mano (no usa el Bottom Tabs Navigator de React Navigation), reutilizada en las cuatro pantallas de entrenador. No tiene estado propio ni hooks: recibe activeScreen (para saber qué pestaña resaltar), isLandscape (para ajustar su altura) y navigation (para poder navegar) todo por props desde la pantalla que la usa. Cada botón llama navigation.navigate('.') con el nombre exacto de la ruta registrada en el grupo de entrenador del navigator.”